Streamlit Tutorial Series

A Complete Guide for Students

From Hello World to Full Image Classification App

Examples 01 – 12 | Keras Integration | GitHub & Streamlit Cloud Deployment

Table of Contents

1. What is Streamlit?
2. Installation & Setup
3. How Streamlit Works
4. Example 01 – Hello Streamlit
5. Example 02 – Text & Markdown
6. Example 03 – Widgets Basics
7. Example 04 – Layout (Columns, Sidebar, Tabs)
8. Example 05 – Displaying Data
9. Example 06 – Charts & Plots
10. Example 07 – File Upload
11. Example 08 – Session State
12. Example 09 – Forms
13. Example 10 – Multi-Page App
14. Example 11 – Image Display
15. Example 12 – Full Image Classification App
16. Deploying on GitHub & Streamlit Cloud
17. Quick Reference Cheatsheet

1. What is Streamlit?

Streamlit is a free, open-source Python library that turns a plain .py script into a fully interactive web application — without any HTML, CSS, or JavaScript. It is specifically designed for data scientists and machine learning engineers who want to share their work as a live, interactive tool.

With Streamlit you can:

- Build interactive dashboards and data apps in pure Python
- Display charts, tables, images, and model predictions
- Accept user inputs (files, text, sliders, dropdowns)
- Deploy your app online for free in minutes via Streamlit Cloud

Key Facts About Streamlit
• Created by: Streamlit Inc. (now part of Snowflake)
• Language: Python only — no web development knowledge required
• Execution model: the entire script reruns top-to-bottom on every user interaction
• Deployment: one command to run locally; one click to deploy to the cloud
• License: Apache 2.0 — completely free and open source

2. Installation & Setup

2.1 Requirements

- Python 3.8 or higher
- pip (comes with Python)
- A code editor (VS Code recommended)

2.2 Install Streamlit

```
# Open your terminal and run:
pip install streamlit

# Verify the installation:
streamlit --version
```

2.3 Install All Required Packages

For the full tutorial (including image classification):

```
pip install streamlit pandas numpy matplotlib seaborn
pip install Pillow scikit-learn tensorflow opencv-python-headless
```

2.4 Your First Run

```
# Create a file called app.py, then run:
streamlit run app.py

# Streamlit opens your browser automatically at:
# http://localhost:8501
```

3. How Streamlit Works

Understanding the Streamlit execution model is the most important concept in this tutorial.

3.1 The Rerun Model

Every time the user interacts with the app (clicks a button, moves a slider, types text), Streamlit RERUNS the entire Python script from top to bottom. This means:

- You write your app like a normal Python script — top to bottom
- Every widget call (st.button, st.slider, etc.) returns the current value
- You use if statements to react to widget values
- There is no need for callbacks or event handlers

```
import streamlit as st

# This runs every time anything changes
name = st.text_input("Enter your name")

# React immediately to the input
if name:
    st.write(f"Hello, {name}!")
```

3.2 Session State

Because the script reruns completely on every interaction, regular Python variables are reset each time. To remember a value across reruns, use st.session_state:

```
# Initialize once
if "counter" not in st.session_state:
    st.session_state.counter = 0

# Update on button click
if st.button("Click me"):
    st.session_state.counter += 1

# Read the persisted value
st.write(f"Count: {st.session_state.counter}")
```

3.3 Multi-Page Apps

Streamlit automatically detects a pages/ folder next to your app.py and adds navigation links in the sidebar for every .py file it finds there:

```
my_app/
├── app.py          <- run this file
└── pages/
    ├── 1_Settings.py
    ├── 2_Data.py
    └── 3_Results.py
```

4. Example 01. Hello Streamlit

Learning Objectives

- Understand what Streamlit is and how to run it
- Display a title, header, subheader, and text on the page
- Run a Streamlit app from the terminal

Why We Need This

Before building anything complex, students need to see the simplest possible Streamlit app to understand the structure: import the library, call functions that start with st., and run the script.

How It Works

Every Streamlit command starts with st. followed by the function name.

st.title() creates the biggest heading. st.header() and st.subheader() create smaller headings. st.write() is the all-purpose display function.

Run any Streamlit app with: streamlit run filename.py

Preview

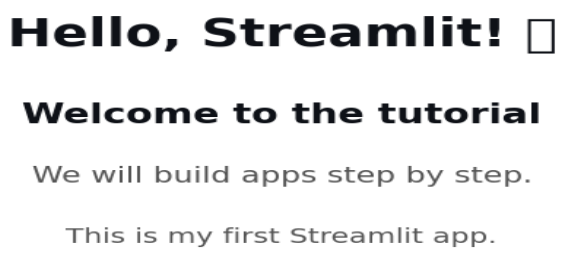


Figure: Hello Streamlit — browser output

Complete Code

```
import streamlit as st

# Biggest heading on the page
st.title("Hello, Streamlit!")

# Section heading
st.header("Welcome to the tutorial")

# Smaller heading
st.subheader("We will build apps step by step.")

# General purpose text display
st.write("This is my first Streamlit app.")
```

Key Functions

Function	What it does
st.title("text")	Biggest heading on the page
st.header("text")	Section heading
st.subheader("text")	Smaller subheading
st.write("text")	General purpose — displays text, numbers, DataFrames

Student Exercise

- Change the title to your own name
- Add a subheader with today's date
- Add st.write() with a short introduction about yourself

---

5. Example 02. Text & Markdown

Learning Objectives

- Format text using Markdown (bold, italic, lists, headers)
- Add colored text using HTML inside Markdown
- Display code blocks with syntax highlighting
- Use special message boxes: info, success, warning, error

Why We Need This

Every app needs to communicate with the user through text. Knowing how to format text makes apps clear and professional. Streamlit supports Markdown — the same format used in Jupyter notebooks.

How It Works

st.markdown() accepts standard Markdown syntax: **bold**, *italic*, ## heading, - bullet.

Passing unsafe_allow_html=True allows real HTML tags inside Markdown for custom colors.

st.code() displays syntax-highlighted code. st.info(), st.success(), st.warning(), st.error() show colored alert boxes.

Preview

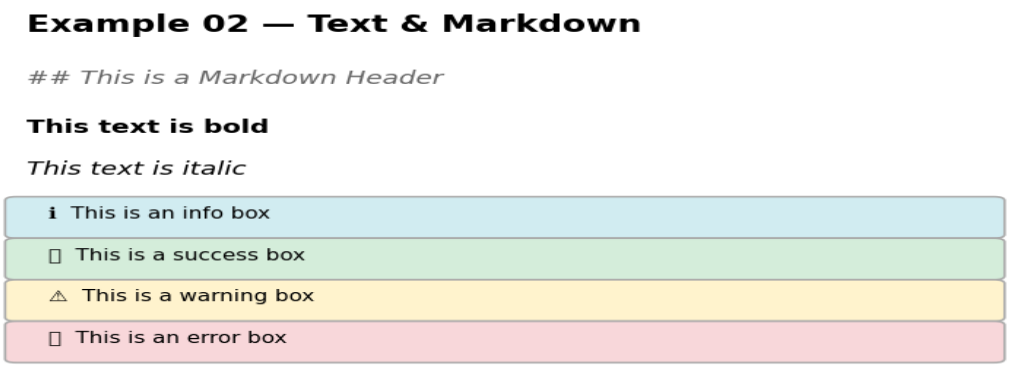


Figure: Text & Markdown — browser output

Complete Code

```
import streamlit as st

st.title("Example 02 - Text & Markdown")

# Markdown formatting
st.markdown("**This text is bold**")
st.markdown("*This text is italic*")
st.markdown("## Section Header")

# Bullet list
st.markdown("""
### My List
- Item one
- Item two
- Item three
""")

# Colored text using HTML
st.markdown("<p style='color: red;'>Red text</p>",
            unsafe_allow_html=True)

# Code block
st.code("import streamlit as st", language="python")

# Message boxes
st.info("This is an info box")
st.success("This is a success box")
st.warning("This is a warning box")
st.error("This is an error box")
```

Key Functions

Function	**What it does**
st.markdown("text")	Formatted Markdown text
st.code("code", language="python")	Syntax-highlighted code block
st.info("text")	Blue info box
st.success("text")	Green success box
st.warning("text")	Yellow warning box
st.error("text")	Red error box

Student Exercise

- Add your name as a large colored heading
- Write a numbered list of 3 things you learned today

- Display a Python code snippet of your choice
- Add a green success box that says "I completed Example 02!"

---

6. Example 03. Widgets Basics

Learning Objectives

- Add interactive buttons, sliders, and text inputs
- Use checkboxes, radio buttons, and dropdown menus
- Read widget values and react to them with if statements
- Build a mini profile card app using all widgets

Why We Need This

Widgets are the interactive building blocks of any Streamlit app. They let users control what the app does — choosing values, typing text, or clicking buttons. Every real app is built from these elements.

How It Works

Every widget is a single Python function call that returns the current value.

The pattern is always: variable = st.widget_name("label", ...options)

When the user changes a widget, the script reruns and the variable has the new value automatically.

Preview

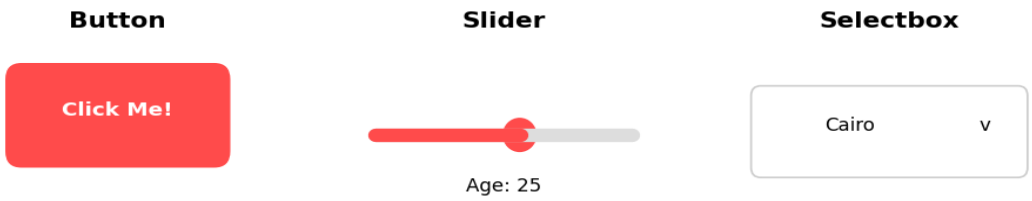


Figure: Widgets Basics — browser output

Complete Code

```
import streamlit as st

# Button — returns True when clicked
if st.button("Click Me!"):
    st.success("You clicked the button!")

# Slider — returns the selected number
age = st.slider("Select your age", 0, 100, 25)
st.write(f"You selected: {age} years old")

# Text input
name = st.text_input("Enter your name")
if name:
    st.success(f"Hello, {name}!")

# Checkbox
agree = st.checkbox("I agree to the terms")

# Radio buttons
subject = st.radio("Favourite subject",
    ["Math", "Science", "CS"])

# Selectbox (dropdown)
city = st.selectbox("Select your city",
    ["Cairo", "Dubai", "London"])

st.write(f"City: {city}, Subject: {subject}")
```

Key Functions

Function	**What it does**
st.button("label")	Returns True when clicked
st.slider("label", min, max, default)	Number picker via drag
st.text_input("label")	Single-line text box
st.text_area("label")	Multi-line text box
st.checkbox("label")	Returns True/False
st.radio("label", [options])	Pick one from a list
st.selectbox("label", [options])	Dropdown menu
st.number_input("label", min, max)	Precise number entry

Student Exercise

- Build a BMI Calculator: sliders for height and weight, button to calculate
- Formula: BMI = weight / (height/100)^2
- Show the result and a colored message box based on BMI category

7. Example 04. Layout — Columns, Sidebar, Tabs

Learning Objectives

- Arrange content side by side using columns
- Move controls to a sidebar panel
- Hide/show content with expanders
- Organize content into clickable tabs
- Display key numbers with metric cards

Why We Need This

Without layout tools, everything appears in one single column from top to bottom. Layout tools let you build professional-looking apps where controls are on the side, results appear next to each other, and related content is grouped into tabs.

How It Works

col1, col2 = st.columns(2) creates two equal side-by-side columns. Use with col1: to put content inside each.

Everything inside with st.sidebar: appears in the left panel of the browser.

st.tabs(["A","B","C"]) creates clickable tab sections. st.expander("label") creates a collapsible section.

Preview

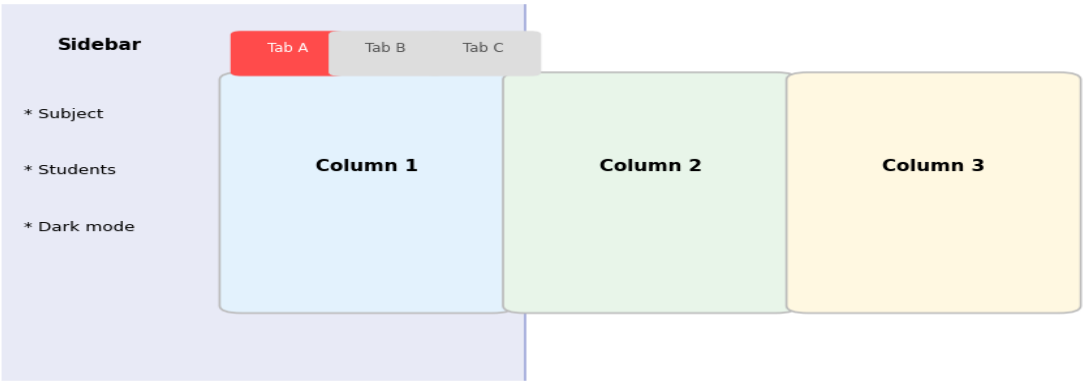


Figure: Layout — Columns, Sidebar, Tabs — browser output

Complete Code

```
import streamlit as st

# Side-by-side columns
col1, col2, col3 = st.columns(3)
with col1:
    st.metric("Students", 120)
with col2:
    st.metric("Average", "87.5", delta="+3.2")
with col3:
    st.metric("Pass Rate", "94%")

# Sidebar
with st.sidebar:
    st.header("Settings")
    subject = st.selectbox("Subject", ["Math","Science"])
    students = st.slider("Students", 10, 100, 30)

# Tabs
tab1, tab2 = st.tabs(["Results", "Details"])
with tab1:
    st.write("Results tab content")
with tab2:
    st.write("Details tab content")

# Expander
with st.expander("Click to see more"):
    st.write("Hidden content revealed!")
```

Key Functions

Function	**What it does**
st.columns(n)	Creates n side-by-side columns
st.columns([1,2,1])	Columns with custom widths
st.sidebar	Left-side panel for controls
st.tabs(["A","B"])	Clickable tab sections
st.expander("label")	Collapsible section
st.container()	Invisible grouping box
st.metric(label, value, delta)	Dashboard number card

Student Exercise

- Build a Student Report Card app
- Put student name, grade, and subject in the sidebar
- Show 4 metric cards: Score, Rank, Attendance, Grade
- Use 3 tabs: Results / Comments / Settings

8. Example 05. Displaying Data

Learning Objectives

- Display pandas DataFrames as interactive sortable tables
- Use metric cards for key numbers
- Filter data interactively with sidebar widgets
- Display JSON and dictionary data

Why We Need This

Almost every real app needs to show data — student scores, image dataset info, model results. Streamlit has powerful built-in tools to display data beautifully without any HTML.

How It Works

st.dataframe() shows an interactive table that the user can sort and scroll.

st.table() shows a static fixed table. st.metric() shows a single number with a label.

Combine widgets with dataframes to let users filter data interactively.

Preview

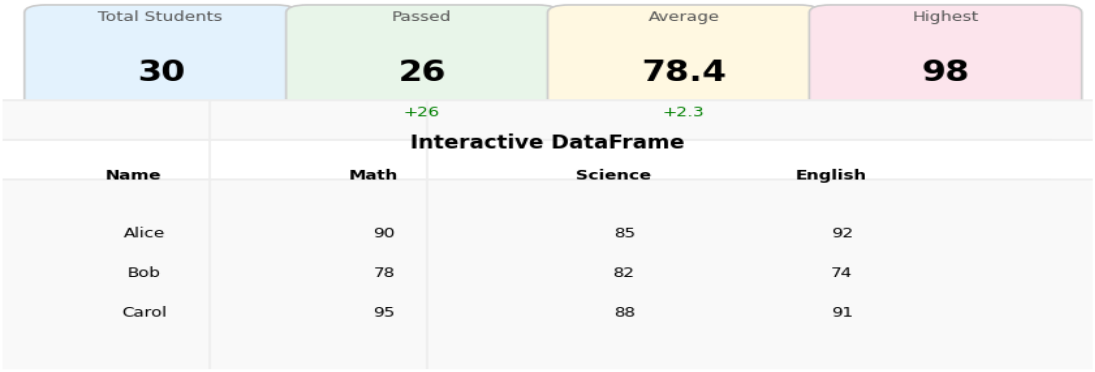


Figure: Displaying Data — browser output

Complete Code

```
import streamlit as st
import pandas as pd
import numpy as np

# Create sample data
df = pd.DataFrame({
    "Name"   : ["Alice", "Bob", "Carol"],
    "Math"   : [90, 75, 88],
    "Science": [85, 80, 92],
    "Grade"  : ["A", "B", "A"],
})

# Interactive sortable table
st.dataframe(df, use_container_width=True)

# Metric cards
col1, col2, col3 = st.columns(3)
with col1:
    st.metric("Total Students", len(df))
with col2:
    st.metric("Average Math", df["Math"].mean())
with col3:
    st.metric("Top Score", df["Math"].max())

# Filter with multiselect
grades = st.multiselect("Filter by Grade",
    df["Grade"].unique(), default=["A","B"])
filtered = df[df["Grade"].isin(grades)]
st.dataframe(filtered)
```

Key Functions

Function	What it does
st.dataframe(df)	Interactive sortable table
st.table(df)	Static clean table
st.metric(label, value, delta)	Single number card with change
st.json(dict)	Pretty-printed dictionary / JSON
st.multiselect("label", options)	Dropdown with multiple selections

Student Exercise

- Create a DataFrame of 15 students with 4 subjects
- Show 4 metric cards: total students, highest score, lowest score, class average
- Add a sidebar slider to filter by minimum score
- Add a multiselect to filter by grade (A, B, C)

9. Example 06. Charts & Plots

Learning Objectives

- Use Streamlit built-in charts (line, bar, area)
- Display Matplotlib and Seaborn figures
- Connect charts to widgets so they update live
- Plot training accuracy and loss curves

Why We Need This

Numbers in a table are hard to understand at a glance. Charts make patterns visible immediately. In our image classification project, we need charts to show training accuracy curves, loss curves, and model comparison bars.

How It Works

Streamlit has built-in chart functions (st.line_chart, st.bar_chart) that need only one line of code.

For full control, create a Matplotlib figure then pass it to st.pyplot(fig).

The same pattern works for Seaborn: create fig, draw on it, then call st.pyplot(fig).

Preview

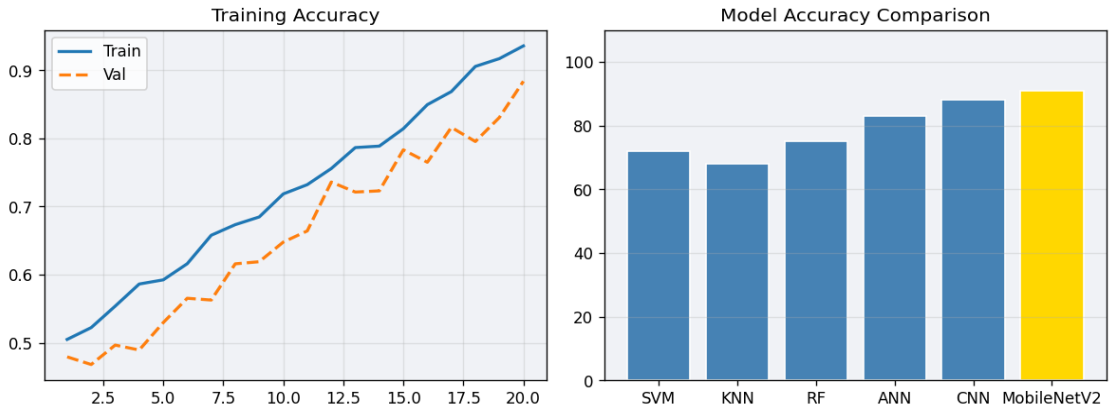


Figure: Charts & Plots — browser output

Complete Code

```
import streamlit as st
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Built-in line chart (one line of code)
history = pd.DataFrame({
    "Train Accuracy": [0.5, 0.7, 0.85, 0.91, 0.94],
    "Val Accuracy"  : [0.48, 0.65, 0.80, 0.87, 0.90],
})
st.line_chart(history)

# Matplotlib figure (full control)
fig, ax = plt.subplots(figsize=(8, 4))
models = ["SVM", "KNN", "RF", "ANN", "CNN", "MobileNetV2"]
accs   = [72, 68, 75, 83, 88, 91]
ax.bar(models, accs, color="steelblue")
ax.set_title("Model Accuracy Comparison")
ax.set_ylabel("Accuracy (%)")
ax.set_ylim(0, 110)
ax.grid(axis="y", alpha=0.3)
plt.tight_layout()
st.pyplot(fig)
```

Key Functions

Function	What it does
st.line_chart(df)	Quick line chart from DataFrame
st.bar_chart(df)	Quick bar chart
st.area_chart(df)	Quick area/filled chart
st.pyplot(fig)	Display any Matplotlib/Seaborn figure

Student Exercise

- Create fake training history for 3 models: ANN, CNN, MobileNetV2
- Show a line chart of all 3 validation accuracies on one chart
- Add a selectbox to switch between Accuracy and Loss views
- Add a slider to control how many epochs to display

10. Example 07. File Upload

Learning Objectives

- Let users upload a CSV file and display it as a table
- Let users upload an image and show it
- Read uploaded files into pandas or NumPy
- Show file information (name, size, type)
- Accept multiple files at once

Why We Need This

In our image classification app, the user needs to upload an image to get a prediction. File upload is one of the most important Streamlit features for real-world apps. Without it, users cannot bring their own data.

How It Works

st.file_uploader() returns a file object when a file is uploaded, or None when nothing is uploaded.

Always check if uploaded_file is not None: before trying to read the file.

For images, pass the file object to PIL.Image.open(). For CSV, pass it to pd.read_csv().

Preview

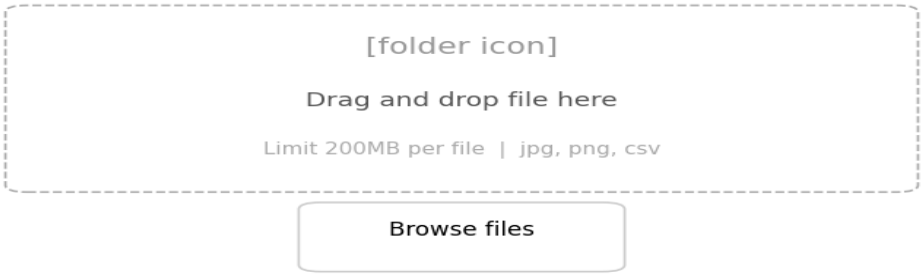


Figure: File Upload — browser output

Complete Code

```
import streamlit as st
import pandas as pd
import numpy as np
from PIL import Image

# Upload a CSV file
csv_file = st.file_uploader("Upload CSV", type=["csv"])
if csv_file is not None:
    df = pd.read_csv(csv_file)
    st.success(f"Loaded {len(df)} rows")
    st.dataframe(df.head())

# Upload an image
img_file = st.file_uploader("Upload image",
    type=["jpg", "jpeg", "png"])
if img_file is not None:
    image = Image.open(img_file)
    img_array = np.array(image)
    st.image(image, caption="Uploaded Image", width=300)
    st.write(f"Size: {image.width} x {image.height} px")
    st.write(f"Shape: {img_array.shape}")

# Multiple files
files = st.file_uploader("Multiple images",
    type=["jpg","png"], accept_multiple_files=True)
if files:
    cols = st.columns(3)
    for i, f in enumerate(files):
        cols[i%3].image(Image.open(f), use_column_width=True)
```

Key Functions

Function	What it does
st.file_uploader("label", type=[...])	File upload widget
accept_multiple_files=True	Allow selecting multiple files
Image.open(file)	Open uploaded image with PIL
pd.read_csv(file)	Read uploaded CSV into DataFrame
np.array(image)	Convert PIL image to NumPy array
st.image(image, width=300)	Display an image on the page

Student Exercise

- Build an Image Info App: upload an image and show width, height, channels, file size
- Add a slider to resize the display width (100 to 800 px)
- Add a checkbox "Convert to Grayscale" and show the grayscale version if checked

11. Example 08. Session State

Learning Objectives

- Understand why Streamlit reruns the script on every interaction
- Use st.session_state to remember values between reruns
- Build a counter that actually works
- Build a to-do list app
- Build a multi-step wizard form

Why We Need This

The problem: every time the user clicks a button or moves a slider, Streamlit reruns the entire script. Any variable you created is reset. Session state is the solution — it is a dictionary that survives reruns.

How It Works

Initialize a key with: if "key" not in st.session_state: st.session_state.key = default_value

Read it with: st.session_state.key

Update it with: st.session_state.key = new_value

Force an immediate rerun with: st.rerun()

Preview

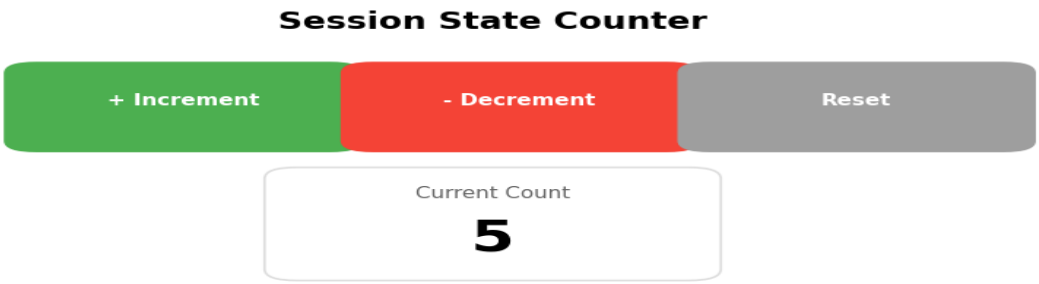


Figure: Session State — browser output

Complete Code

```
import streamlit as st

# Step 1: Initialize the counter once
if "counter" not in st.session_state:
    st.session_state.counter = 0

# Step 2: Buttons to change the value
col1, col2, col3 = st.columns(3)
with col1:
    if st.button("+ Increment"):
        st.session_state.counter += 1
with col2:
    if st.button("- Decrement"):
        st.session_state.counter -= 1
with col3:
    if st.button("Reset"):
        st.session_state.counter = 0

# Step 3: Display the persisted value
st.metric("Current Count", st.session_state.counter)

# To-do list example
if "tasks" not in st.session_state:
    st.session_state.tasks = []

new_task = st.text_input("New task")
if st.button("Add Task") and new_task:
    st.session_state.tasks.append(new_task)

for task in st.session_state.tasks:
    st.write(f"* {task}")
```

Key Functions

Function	**What it does**
st.session_state.key = value	Store a value that survives reruns
st.session_state.key	Read the stored value
"key" not in st.session_state	Check if key exists before initializing
st.rerun()	Force the script to rerun immediately
key="unique_id"	Give each widget a unique key when needed

Student Exercise

- Build a Quiz App with 3 questions
- Show one question at a time using session state

- Track the score in session state
- Show the final score and a Restart button at the end

---

12. Example 09. Forms

Learning Objectives

- Group multiple widgets inside a form
- Submit all inputs at once with one button
- Prevent the app from rerunning on every small change
- Build a login form, search form, and model config form

Why We Need This

Without a form, the app reruns every time the user touches any widget — even while they are typing. A form collects all inputs and only submits when the user clicks Submit. This is cleaner and more efficient for registration forms, search queries, and settings panels.

How It Works

Wrap your widgets inside with st.form("unique_name").

Add st.form_submit_button("label") as the last item inside the form.

Nothing inside the form triggers a rerun until the submit button is clicked.

Handle the result outside the form block using if submitted:.

Preview



Figure: Forms — browser output

Complete Code

```
import streamlit as st

# Basic registration form
with st.form("registration_form"):
    st.subheader("Student Registration")
    name    = st.text_input("Full Name")
    age     = st.number_input("Age", 15, 60, 20)
    major   = st.selectbox("Major",
        ["AI", "CS", "Engineering"])
    agree   = st.checkbox("I agree to the terms")
    submitted = st.form_submit_button("Register")

# Handle the result OUTSIDE the form
if submitted:
    if not name.strip():
        st.error("Please enter your name")
    elif not agree:
        st.error("Please agree to the terms")
    else:
        st.success(f"Welcome, {name}!")

# Login form with password
with st.form("login_form"):
    username = st.text_input("Username")
    password = st.text_input("Password",
        type="password")  # hides the text
    login = st.form_submit_button("Login")

if login:
    if username == "admin" and password == "1234":
        st.success("Login successful!")
    else:
        st.error("Wrong credentials")
```

Key Functions

Function	What it does
with st.form("name"):	Create a form — nothing inside reruns until submit
st.form_submit_button("label")	The submit button — must be inside the form
type="password"	Hides text input — for passwords
if submitted:	Handle result OUTSIDE the form block

Student Exercise

- Build a BMI Calculator Form with name, height, weight, gender
- Only calculate when Submit is clicked
- Show BMI value and a colored message box based on category

- Categories: Underweight < 18.5 | Normal 18.5-25 | Overweight 25-30 | Obese > 30

---

13. Example 10. Multi-Page App

Learning Objectives

- Split a Streamlit app into multiple pages
- Share data between pages using session state
- Understand how automatic sidebar navigation works
- Build a 3-page Student Portal app

Why We Need This

Real applications are too large to fit in one file. Streamlit automatically creates sidebar navigation for every .py file in a pages/ folder. Each page is a separate file and they share data through session state.

How It Works

Create a pages/ folder next to your app.py. Any .py file inside it becomes a page automatically.

Number the files to control order: 1_Settings.py, 2_Data.py, 3_Report.py

Share data between pages by storing it in st.session_state on one page and reading it on another.

IMPORTANT: Always run streamlit run app.py — never run a page file directly.

Preview

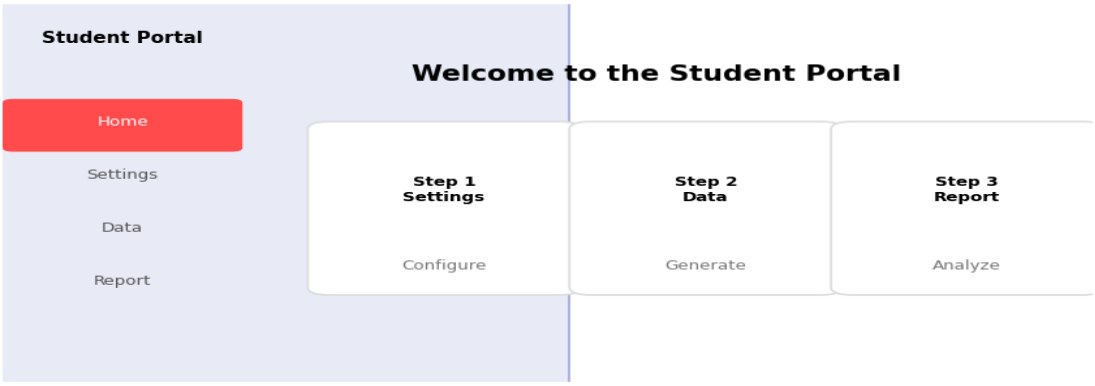


Figure: Multi-Page App — browser output

Complete Code

```
# Folder structure:
# example_10/
# |-- app.py              <- run this
# `-- pages/
#     |-- 1_Settings.py
#     |-- 2_Data.py
#     `-- 3_Report.py

# app.py (Home page)
import streamlit as st

# Initialize shared variables
if "subject" not in st.session_state:
    st.session_state.subject = "Mathematics"
if "num_students" not in st.session_state:
    st.session_state.num_students = 30

st.title("Student Portal")
st.write("Use the sidebar to navigate.")

# pages/1_Settings.py
import streamlit as st

with st.form("settings"):
    subject = st.selectbox("Subject",
        ["Mathematics","Science","English"])
    students = st.slider("Students", 5, 100, 30)
    saved = st.form_submit_button("Save")

if saved:
    # Write to session state -> visible on other pages
    st.session_state.subject = subject
    st.session_state.num_students = students
    st.success("Saved!")
```

Key Functions

Function	What it does
pages/ folder	Put .py files here to create pages automatically
1_Page.py naming	Number prefix controls sidebar order
st.session_state	How pages share data with each other
st.set_page_config(...)	Set page title and icon at the top of each file

Student Exercise

- Create a 3-page app: Settings, Data Generator, Report
- Settings page: pick subject, number of students, passing score
- Data page: generate random scores based on settings, show as table
- Report page: show pass/fail counts, bar chart, written summary

---

14. Example 11. Image Display

Learning Objectives

- Display images from a URL, file upload, and NumPy array
- Show image information: size, channels, pixel range
- Apply image operations: grayscale, flip, rotate, blur, edge detection
- Display images in a grid with column layout
- Download processed images

Why We Need This

This example is the direct bridge to Example 12. Everything here — uploading an image, converting it to a NumPy array, displaying it — is exactly what happens when a user uploads an image to classify in the final app.

How It Works

st.image() accepts a file path, URL, PIL Image object, or NumPy array.

Images in Keras are NumPy arrays with shape (height, width, channels) and values 0-255 or 0.0-1.0.

PIL (Pillow) library provides all image operations: grayscale, flip, rotate, blur, edge detection.

Preview

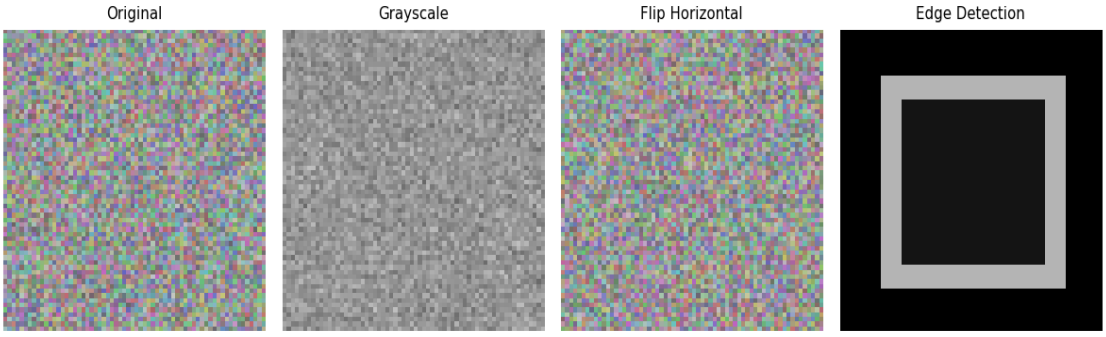


Figure: Image Display — browser output

Complete Code

```
import streamlit as st
import numpy as np
from PIL import Image, ImageFilter, ImageOps

# Display from URL
img_url = "https://upload.wikimedia.org/wikipedia/
           commons/3/3a/Cat03.jpg"
st.image(img_url, caption="Cat from Wikipedia", width=300)

# Upload and display
uploaded = st.file_uploader("Upload image",
    type=["jpg","png"])

if uploaded is not None:
    image = Image.open(uploaded).convert("RGB")
    img_array = np.array(image)
    st.image(image, width=400)
    st.write(f"Shape: {img_array.shape}")
    st.write(f"Range: {img_array.min()} to {img_array.max()}")

    # Apply operations
    op = st.selectbox("Operation",
        ["Grayscale","Flip H","Blur","Edges"])
    if op == "Grayscale":
        result = ImageOps.grayscale(image)
    elif op == "Flip H":
        result = ImageOps.mirror(image)
    elif op == "Blur":
        result = image.filter(ImageFilter.GaussianBlur(4))
    elif op == "Edges":
        result = image.filter(ImageFilter.FIND_EDGES)

    col1, col2 = st.columns(2)
    col1.image(image,  caption="Original")
    col2.image(result, caption=op)
```

Key Functions

Function	**What it does**
st.image(image)	Display image — accepts URL, PIL, NumPy
st.image(image, width=300)	Control display width in pixels
Image.open(file).convert("RGB")	Open and convert to RGB
ImageOps.grayscale(img)	Convert to grayscale
ImageOps.mirror(img)	Flip horizontally
img.filter(ImageFilter.GaussianBlur(4))	Apply blur
img.filter(ImageFilter.FIND_EDGES)	Detect edges
np.array(image) / 255.0	Normalize pixels to 0.0–1.0

Student Exercise

- Build an Image Comparison App: upload two images and show side by side
- Apply the same operation to both simultaneously with one selectbox
- Show a metric comparing their sizes (wider, taller, more pixels)

---

15. Example 12 — Full Image Classification App

Learning Objectives

- Upload an image dataset as a ZIP file
- Train 6+ models: SVM, KNN, Random Forest, ANN, CNN, and pretrained models
- Classify a new uploaded image with any trained model
- View full evaluation results: Confusion Matrix, ROC-AUC, Precision-Recall, Metrics Heatmap

App Structure

```
example_12/
|-- app.py                      <- Home + ZIP dataset upload
|-- requirements.txt
`-- pages/
    |-- 1_Upload_Predict.py     <- Upload image -> prediction
    |-- 2_Train_Models.py       <- Train all models
    `-- 3_Results.py            <- All evaluation figures
```

Preview — Evaluation Figures

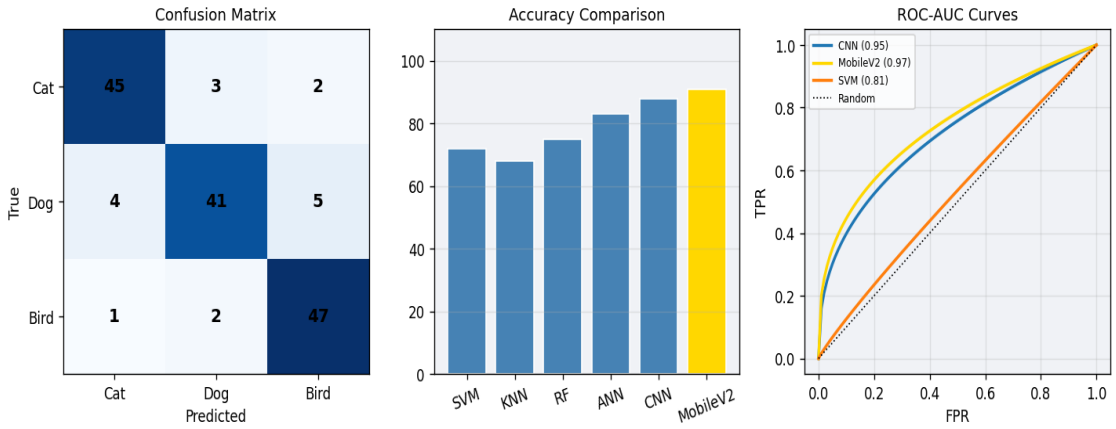


Figure: Confusion Matrix, Accuracy Comparison, and ROC-AUC curves from the Results page

Available Pretrained Models (40 Total)

Model	Input Size | Weights | ImageNet Top-1
MobileNetV2	224px | 14 MB | 71.3%
EfficientNetB0	224px | 29 MB | 77.1%
EfficientNetB3	300px | 48 MB | 81.6%
EfficientNetV2S	384px | 88 MB | 83.9%
ResNet50	224px | 98 MB | 74.9%
ResNet50V2	224px | 98 MB | 76.0%

VGG16	224px | 528 MB | 71.3%
InceptionV3	299px | 92 MB | 77.9%
Xception	299px | 88 MB | 79.0%
DenseNet121	224px | 33 MB | 75.0%
NASNetMobile	224px | 23 MB | 74.4%
ConvNeXtTiny	224px | 109 MB | 81.3%
... 28 more models	Full list on the Train Models page

Results Page — All Figures

Section	What it shows
1. Accuracy Cards	One metric card per model with delta vs best model
2. Accuracy Bar Chart	All models ranked by test accuracy
3. Confusion Matrix	Interactive: raw counts or normalized percentages
4. All Confusion Matrices	Grid showing all models at once
5. ROC-AUC Curves	Per-class + macro average, all models compared
6. Precision-Recall Curves	Per-class Average Precision, mean AP comparison
7. Per-Class Metrics	Grouped bar: Precision / Recall / F1 per class
8. Metrics Heatmap	All models x all metrics in one color-coded table
9. Error Analysis	Misclassification rate, confused pairs, distribution
10. Final Summary Table	Accuracy, F1, ROC-AUC, mAP, Kappa, MCC

Dataset Preparation

Your dataset must be packed as a ZIP file with one subfolder per class:

```
my_dataset.zip
|-- cat/
|    |-- cat001.jpg
|    `-- cat002.jpg
`-- dog/
     |-- dog001.jpg
     `-- dog002.jpg
```

Note: Class folders must be at the TOP LEVEL of the ZIP — not inside an extra subfolder.

16. Deploying on GitHub & Streamlit Cloud

Streamlit Cloud lets you deploy any Streamlit app for FREE from a GitHub repository in minutes. Your app gets a public URL that anyone can open in a browser.

16.1 Step-by-Step Deployment

Step 1 — Create a GitHub Account

- Go to https://github.com and create a free account
- Verify your email address

Step 2 — Install Git

```
# Windows: download from https://git-scm.com
# Mac:
brew install git
# Ubuntu/Linux:
sudo apt install git

# Verify:
git --version
```

Step 3 — Create a New GitHub Repository

- Go to https://github.com/new
- Repository name: e.g. streamlit-tutorial
- Set to Public (required for free Streamlit Cloud deployment)
- Click Create repository

Step 4 — Add requirements.txt to Your Project

This file tells Streamlit Cloud which packages to install. Create it in the same folder as app.py:

```
# requirements.txt
streamlit
pandas
numpy
matplotlib
seaborn
Pillow
scikit-learn
tensorflow
opencv-python-headless
```

Step 5 — Push Your Code to GitHub

```
# Navigate to your project folder
cd my-streamlit-app

# Initialize git
git init

# Add all files
git add .

# Commit
git commit -m "Initial commit: add Streamlit tutorial"

# Connect to your GitHub repository
git remote add origin https://github.com/YOUR_USERNAME/streamlit-tutorial.git

# Push to GitHub
git push -u origin main
```

Step 6 — Deploy on Streamlit Cloud

- Go to https://share.streamlit.io
- Click "Sign in with GitHub" and authorize Streamlit
- Click "New app"
- Select your repository from the dropdown
- Set the Main file path — e.g. app.py or example_12/app.py
- Click "Deploy" and wait 1-2 minutes

Your app is now live at a URL like:

```
https://your-username-streamlit-tutorial-app-xxxx.streamlit.app
```

16.2 Project Structure on GitHub

```
my-streamlit-app/              <- GitHub repository root
|-- app.py                     <- Main entry point
|-- requirements.txt           <- Package list for Streamlit Cloud
|-- README.md                  <- Project description
`-- pages/
    |-- 1_Settings.py
    |-- 2_Data.py
    `-- 3_Results.py

# NEVER upload your dataset folder to GitHub!
# Use .gitignore to exclude it:
```

Step 7 — Create .gitignore

Add this file to your project to prevent accidental uploads of large files:

```
# .gitignore
dataset/
*.zip
*.h5
*.keras
__pycache__/
*.pyc
.env
```

16.3 Updating Your App

Every time you push new code to GitHub, Streamlit Cloud redeploys automatically:

```
# Make changes to your code, then:
git add .
git commit -m "Update: added new feature"
git push

# Streamlit Cloud detects the push and redeploys automatically
# Takes about 1 minute
```

16.4 Fixing the ModuleNotFoundError

If you see a ModuleNotFoundError on Streamlit Cloud, a package is missing from requirements.txt:

```
# Error example:
# ModuleNotFoundError: No module named "matplotlib"

# Fix: add the missing package to requirements.txt
# Then push to GitHub:
git add requirements.txt
git commit -m "Fix: add matplotlib to requirements"
git push

# Streamlit Cloud will reinstall and redeploy automatically
```

16.5 Common Deployment Issues & Fixes

Problem	**Fix**
ModuleNotFoundError	Add the missing package to requirements.txt and push
Pages not showing in sidebar	Ensure pages/ folder is next to app.py in the repo
App crashes on large dataset	Use st.file_uploader instead of local folder path
Slow app on Cloud	Reduce image size or add @st.cache_data decorator
Secret API keys in code	Use Streamlit Secrets Manager instead of hardcoding
App works locally but not Cloud	Check requirements.txt has every import you use

16.6 Local vs Cloud Differences

Feature	**Local vs Streamlit Cloud**
Install packages	pip install X vs Add X to requirements.txt
Dataset location	Local folder vs Upload via st.file_uploader
File storage	Persists vs Resets on each session (use cloud storage)
GPU	Your GPU if available vs CPU only on free tier
Memory	Your RAM vs 1 GB RAM on free tier

17. Quick Reference Cheatsheet

Display Functions

Function	What it shows
st.title("text")	Biggest heading
st.header("text")	Section heading
st.subheader("text")	Smaller section heading
st.write("text or data")	Auto-detect: text, number, DataFrame, chart
st.markdown("**bold**")	Formatted Markdown text
st.code("x=1", language="python")	Syntax-highlighted code block
st.info / success / warning / error	Colored alert boxes
st.image(img, width=300)	Display image (URL, PIL, NumPy)
st.dataframe(df)	Interactive sortable table
st.table(df)	Static clean table
st.metric("label", value)	Dashboard number card
st.json(dict)	Pretty-printed dictionary
st.pyplot(fig)	Display Matplotlib/Seaborn figure

Widget Functions

Widget	Returns
st.button("label")	True when clicked
st.slider("label", min, max, default)	Selected number
st.text_input("label")	Typed string
st.text_area("label")	Multi-line typed string
st.number_input("label", min, max)	Selected number (precise)
st.checkbox("label")	True or False
st.radio("label", [options])	Selected option string
st.selectbox("label", [options])	Selected option string
st.multiselect("label", [options])	List of selected options
st.select_slider("label", [options])	Selected option from list
st.file_uploader("label", type=[...])	File object or None
st.color_picker("label")	Hex color string
st.date_input("label")	datetime.date object

Layout Functions

Function	What it does
col1, col2 = st.columns(2)	Create 2 equal side-by-side columns
st.columns([1, 2, 1])	Columns with relative widths
with st.sidebar:	Content goes to left sidebar panel
tab1, tab2 = st.tabs(["A","B"])	Create clickable tabs
with st.expander("label"):	Collapsible hidden section
st.container()	Invisible grouping box
st.empty()	Placeholder to fill later
st.divider()	Horizontal separator line

Session State

Pattern	What it does
if "key" not in st.session_state: st.session_state.key = value	Initialize once
st.session_state.key	Read stored value
st.session_state.key = new_val	Update stored value
st.rerun()	Force immediate rerun

Multi-Page Apps

Rule	Details
Folder name	Must be exactly pages/ (lowercase)
Location	pages/ must be next to app.py
Run command	Always streamlit run app.py — never a page file
Order control	Name files 1_Page.py, 2_Page.py to control order
Data sharing	Use st.session_state to pass data between pages
Page config	Add st.set_page_config() at the TOP of each page file

Deployment Checklist

- app.py exists at the root of your GitHub repository
- requirements.txt lists every package you import
- pages/ folder is next to app.py (not inside a subfolder)
- .gitignore excludes dataset/ and *.zip
- No local file paths hardcoded (use st.file_uploader instead)
- Repository is set to Public on GitHub